SESSION 2 OF 3

Verilog HDL

Combinational Design

Assign Statement, If-Else, Case Statement & Loop Statement

Assign Statement · If-Else

Case Statement · Loop Statement



SESSION 2 AGENDA

Session 2 Overview

4 sections — building combinational logic with real RTL examples

1. **Assign Statement**
2. **If-Else**
3. **Case Statement**
4. **Loop Statement**

7 · ASSIGN STATEMENT

Assign Statement

A concise continuous assignment for combinational logic

```
wire [3:0] sum;
wire       carry;

assign {carry, sum} = a + b;
assign y = sel ? in1 : in0;
assign led = ~switch;
```

- Used only with wire, never with reg
- Operates 'continuously' — updates instantly whenever the right-hand side changes, just like a real wire
- Doesn't need to be inside an always block — write it directly outside any procedural block
- Best for simple single-line logic, such as gates or small muxes

assign vs always: assign is a shortcut for single-line combinational logic, while always supports more complex logic (including sequential)

7 · ASSIGN STATEMENT (CONT.)

assign vs always: Full Comparison Table

Choosing the right one based on the type of logic

Aspect	assign	always
Left-hand side type	wire only	reg only
Logic type supported	Simple combinational only	Combinational or sequential
Assign lines per signal	1 signal — more than 1 assign causes a multiple driver error	Can update under several conditions in one block
Control statement support	Not supported (only ?:)	Fully supports if/case/for/while
Position in code	Outside any procedural block	Must be inside always/initial

7 · ASSIGN STATEMENT (CONT.)

Half Adder Example

A minimal combinational design built entirely with assign

```
module half_adder (
  input  a, b,
  output sum, carry
);
  assign sum   = a ^ b;   // XOR
  assign carry = a & b;  // AND
endmodule
```

a	b	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

- A half adder adds two single bits and produces a sum bit and a carry bit
- sum is simply XOR of the inputs; carry is simply AND of the inputs
- Both outputs are pure combinational logic — no clock, no memory needed
- This is the simplest possible RTL example using only assign statements

7 · ASSIGN STATEMENT (CONT.)

MUX Using assign vs always

Two different code styles that synthesize into identical hardware

```
// Style A: assign + conditional operator
module mux_assign(
  input  a, b, sel,
  output y
);
  assign y = sel ? b : a;
endmodule
```

```
// Style B: always + if-else
module mux_always(
  input  a, b, sel,
  output reg y
);
  always @(*) begin
    if (sel) y = b;
    else     y = a;
  end
endmodule
```

Same Hardware, Different Style

- Both styles synthesize to exactly the same 2:1 multiplexer — the synthesis tool doesn't care which style you used
- assign is more concise for simple combinational logic; always with if/case scales better to complex logic
- Style choice is a matter of readability and team convention, not a hardware difference

8 · IF-ELSE

If-else Statement

Conditional logic control — synthesizes into a priority mux

```
always @(*) begin
  if (sel == 2'b00)
    y = a;
  else if (sel == 2'b01)
    y = b;
  else if (sel == 2'b10)
    y = c;
  else
    y = d;
end
```

- The first condition has the highest priority — synthesizes into a priority encoder/mux
- Always include an else covering all cases to avoid an unintended inferred latch
- Used only inside an always block — never outside a procedural block

8 · IF-ELSE (CONT.)

If-else vs Case: When to Use Each?

Comparing how each one is synthesized

Aspect	If-else	Case
Priority	Yes — the first condition matters most	No — every branch is equal
Resulting hardware	Priority encoder/mux (slower with many conditions)	Parallel mux (faster if cases don't overlap)
Best for	Complex conditions, comparing ranges of values	Selecting from a fixed set, e.g. opcode, state
Risk	Inferred latch if no else	Inferred latch if no default

General rule: use case for opcode/state encoding; use if-else when comparing ranges or when priority matters

8 · IF-ELSE (CONT.)

Priority Encoder Example

A classic real-world use of if-else priority logic

```
module priority_encoder (
  input  [3:0] req,
  output reg [1:0] grant_id,
  output reg valid
);
  always @(*) begin
    valid = 1'b1;
    if (req[3])      grant_id = 2'd3;  // highest priority
    else if (req[2]) grant_id = 2'd2;
    else if (req[1]) grant_id = 2'd1;
    else if (req[0]) grant_id = 2'd0;  // lowest priority
    else begin
      grant_id = 2'd0;
      valid    = 1'b0;  // no request active
    end
  end
endmodule
```

Why Priority Matters

- req[3] is checked first, so it always wins when multiple requests are active simultaneously
- The else branch handles the 'no request' case and sets valid = 0
- This is the same priority concept used in interrupt controllers and bus arbiters
- If-else (not case) is the natural fit here because priority order matters

9 · CASE STATEMENT

Case Statement

A more readable alternative to if-else for multiple conditions

```
always @(*) begin
  case (opcode)
    3'b000: result = a + b;
    3'b001: result = a - b;
    3'b010: result = a & b;
    3'b011: result = a | b;
    default: result = 0;
  endcase
end
```

- Every branch in a case has equal priority (unlike if-else)
- default prevents inferred latches — always include it
- casex/casez support don't-care matching (x or z), commonly used for priority decoding

9 · CASE STATEMENT (CONT.)

casex and casez

Don't-care matching for priority decoding

```
// casez: z = don't care
always @(*) begin
  casez (interrupt)
    4'b1???: priority = 3; // bit3=1
    4'b01??: priority = 2;
    4'b001?: priority = 1;
    4'b0001: priority = 0;
    default: priority = 4;
  endcase
end
```

- casez treats z (high-impedance) as don't-care during comparison
- casex treats both x and z as don't-care — riskier for simulation/synthesis mismatches
- Commonly used for priority interrupt decoders or instruction decoders with many patterns
- Avoid casex in synthesizable code, since x in real hardware behaves differently than in simulation

9 · CASE STATEMENT (CONT.)

ALU Example

A 4-function Arithmetic Logic Unit built with a case statement

```
module alu (
  input  [7:0] a, b,
  input  [1:0] op,
  output reg [7:0] result,
  output zero
);
  always @(*) begin
    case (op)
      2'b00: result = a + b;   // ADD
      2'b01: result = a - b;   // SUB
      2'b10: result = a & b;   // AND
      2'b11: result = a | b;   // OR
      default: result = 8'b0;
    endcase
  end
  assign zero = (result == 8'b0);
endmodule
```

Design Notes

- op selects which of 4 operations to perform — a textbook case for case
- result is combinational, so it updates immediately whenever a, b, or op changes
- zero is a status flag, computed separately with assign
- This is the basic building block of a CPU's execution unit

10 · LOOP STATEMENT

Loop Statements

4 ways to repeat execution inside a procedural block

a. While

b. For loop

c. Repeat

d. Forever

- Usable only inside always or initial blocks (procedural code)
- for and repeat are synthesizable when the iteration count is fixed; while/forever are typically simulation-only
- Choose the loop type that best fits the iteration pattern you need

10 · LOOP STATEMENT

While Loop

Repeats as long as a condition is true — commonly used in testbenches

```
integer i;
initial begin
  i = 0;
  while (i < 8) begin
    $display("i = %d", i);
    i = i + 1;
  end
end
```

- Frequently used in testbenches to loop until a condition changes
- Synthesizing a while loop requires a fixed bound on iterations, otherwise it cannot be synthesized
- Watch out for infinite loops if the condition never becomes false

10 · LOOP STATEMENT

For Loop

Repeats a fixed number of times — synthesizes well

```
integer k;
reg [7:0] parity;
always @(*) begin
  parity = 0;
  for (k = 0; k < 8; k = k + 1)
    parity = parity ^ data[k];
end
```

- Well suited to fixed-iteration patterns, e.g. parity checks, unrolled adders
- Synthesis tools 'unroll' a for loop into a parallel circuit based on the iteration count
- Must always have a clear initial value, condition, and increment

10 · LOOP STATEMENT (CONT.)

Example: Using a For Loop for Memory Init

A pattern commonly seen in verification and testbenches

```
reg [7:0] mem [0:15]; // 16 x 8-bit memory
integer idx;

initial begin
  for (idx = 0; idx < 16; idx = idx + 1)
    mem[idx] = idx * 2;
end

// synthesizable version: unrolled adder
always @(*) begin
  sum = 0;
  for (idx = 0; idx < 8; idx = idx + 1)
    sum = sum + data[idx];
end
```

- First block: used only in testbenches (not synthesizable — it's a behavioral memory model)
- Second block: actually synthesizable — the tool unrolls it into 8 parallel adders
- The loop variable (idx) must always be declared as an integer outside the loop

10 · LOOP STATEMENT (CONT.)

Parity Generator

Visualizing how a for loop unrolls into real hardware

```
module parity_gen (
  input  [7:0] data,
  output reg   parity
);
  integer i;
  always @(*) begin
    parity = 1'b0;
    for (i = 0; i < 8; i = i + 1)
      parity = parity ^ data[i];
  end
endmodule
```

What Synthesis Produces

- The for loop does NOT become a real loop in hardware — there's no 'looping circuit'
- Synthesis unrolls it into a chain of 7 XOR gates wired together (a tree or chain structure)
- Loop iteration count must be a compile-time constant for this to work
- This is the key mental model: for loops describe repeated structure, not runtime iteration

data[0] ^ data[1] ^ data[2] ^ ... ^ data[7] → parity

10 · LOOP STATEMENT

Repeat Statement

Repeats a fixed, predetermined number of times

```
initial begin
  repeat (4) begin
    @(posedge clk);
    $display("clock tick");
  end
end
```

- repeat (N) executes a fixed N times, with no condition like while
- Commonly used in testbenches to wait for a specific number of clock cycles
- Unlike a for loop, it has no built-in loop variable

Session 2 — Summary

- Assign Statement — continuous assignment, Half Adder example, assign vs always comparison ✓
- If-Else — priority logic, comparison with case, Priority Encoder example ✓
- Case Statement — parallel logic, casex/casez, ALU example ✓
- Loop Statement — while, for, repeat; how for-loops unroll into hardware, Parity Generator example ✓

Coming next: Session 3 — Sequential Design & Reusable Code (Always Block, Blocking/Non-blocking, Initial, Function, Task)